

Technical Report

A Comprehensive Reevaluation of Large Neighborhood Search in Multi-Agent Path finding: New Findings and Opportunities

Anonymous submission

1 Algorithm Details on RandomWalkProb

RandomWalkProb shares the same motivation as the RandomWalk strategy, which focuses on improving high-delayed agents. However, the original algorithm has two potential limitations related to the selection of `random_walk` search start agents: 1) RandomWalk uses a tracking set to record previously selected high-delayed start agents, preventing them from being selected again for subsequent `random_walk` searches even if they still have significant delays. 2) When the neighborhood size is not reached after a `random_walk` search on the selected high-delayed start agent, a random agent is chosen as the next start agent, which is less informative regarding the delays. Thus, we conduct two modifications to RandomWalk to solve this issue. First, we remove the tracking set. Second, whenever an agent is required to start a `random_walk` search, the agent is sampled according to a probability linear to its delay, i.e. $\Pr(a_i) = \frac{\text{delay}(p_i)}{\sum_j \text{delay}(p_j)}$.

Algo. 1 is the pseudo code comparing RandomWalk and our RandomWalkProb strategy (gray lines only exist in RandomWalk, and blue lines only exist in our RandomWalkProb). Specifically, the potential limitations of the original RandomWalk strategy arise from two key aspects: 1) RandomWalk uses a `tabuList` to track previously selected start agents [Line 2 to Line 3], which is only reset after all delayed agents have been chosen as start agents [Line 4], preventing agents from being reselected. thus agents to be select again. 2) When the neighborhood size is not reached after one search, a random agent is chosen from $\tilde{A}$ as the start agent [Line 9] to perform another `random_walk` search which is less related to the delay. Our RandomWalkProb improves the selection of start agent a_k by sampling agent based on a delay-proportional probability every time [Line 1 and Line 8].

After selecting the start agent a_k , both original RandomWalk and our RandomWalkProb use the same `random_walk()` search function [Line 7; Line 9 to Line 18], which is identical to the `RANDOMWALK()` function in (Li et al. 2021) (Algorithm 1: Line 10 to Line 18), to prioritize moving the start agent a_k along a shorter path. The `random_walk()` function first randomly selects a start state x along agent a_k 's path p_k [Line 11]. It then collects possible vertices $v \in N_x$ where agent a_k at time step $t+1$ can reach and move towards a shorter path to g_k while ignoring

Algorithm 1: RandomWalk / RandomWalkProb

Input: Graph $G = (V, E)$, agents $A = \{a_1, \dots, a_n\}$, neighborhood size M , paths $P = \{p_1, \dots, p_n\}$, `tabuList` from previous LNS iteration, number of delayed agents N_{delay}

- 1 $a_k \sim \Pr(a_k) = \frac{\text{delay}(p_k)}{\sum_i \text{delay}(p_i)}$
- 2 $a_k \leftarrow \arg \max_{a_i \in A \setminus \text{tabuList}} \{\text{delay}(p_i)\}$
- 3 `tabuList` $\leftarrow$ `tabuList` $\cup \{a_k\}$
- 4 **if** $|\text{tabuList}| = N_{\text{delay}}$ **then** `tabuList` $\leftarrow \emptyset$
- 5 $\tilde{A} \leftarrow \{a_k\}$
- 6 **while** $|\tilde{A}| < M$ **do**
- 7 $\tilde{A} \leftarrow \text{random_walk}(G, a_k, P, \tilde{A}, M)$
- 8 $a_k \sim \Pr(a_k) = \frac{\text{delay}(p_k)}{\sum_i \text{delay}(p_i)}$
- 9 $a_k \leftarrow$ random agent in $\tilde{A}$
- 10 **Function** `random_walk` ($G, a_k, P, \tilde{A}, M$) :
- 11 $(x, t) \leftarrow (p_k[t], t)$, where t is a random timestep of p_k
- 12 $N_x \leftarrow \{v \in V | (x, v) \in E \cup \{(x, x)\} \wedge t + 1 + d(v, g_k) < l(p_k)\}$
- 13 **while** $|N_x| > 0 \wedge |\tilde{A}| < M$ **do**
- 14 $y \leftarrow$ a random vertex in N_x
- 15 $\tilde{A} \leftarrow \tilde{A} \cup \{ \text{agents collide with action 'moving to } y' \}$
- 16 $(x, t) \leftarrow (y, t + 1)$
- 17 $N_x \leftarrow \{v \in V | (x, v) \in E \cup \{(x, x)\} \wedge t + 1 + d(v, g_k) < l(p_k)\}$
- 18 **return** $\tilde{A}$

other agents [Line 12]. Since any path passing through vertex v at time step $t + 1$ must be at least $t + 1 + d(v, g_k)$ in length, the condition for selecting v is $t + 1 + d(v, g_k) < l(p_k)$. As long as N_x is not empty and collected agents are not enough [Line 13], the agent moves to a random vertex $y \in N_x$ [Line 14] and adds any agents who collide with action to $\tilde{A}$ [Line 15] and updates the set N_x [Line 17].

2 Additional Details on the Unified Setting

The **initial solutions, replan solvers, and neighborhood sizes** used for different methods are summarized in Table 1. The number of agents evaluated in different maps are summarized in Table 2.

Table 1: MAPF Benchmark Evaluation Settings. 'La' and 'LN' are short for LaCAM2 (Okumura 2023) and LNS2 (Li et al. 2022). PP is Prioritized Planning (Erdmann and Lozano-Perez 1987). PBS is Parallel Push and Swap (Sajid, Luna, and Bekris 2012). EECBS is Explicit Estimation CBS (EECBS) (Li, Ruml, and Koenig 2021). Prefix "Our-" refers methods in our setting. Prefix "Ori-" refers methods in their original settings.

Method	Initial Solution	Replan Solver	Neighborhood size
RandomWalk	La, LN	PP	{4, 8, 16, 32}
Intersection	La, LN	PP	{4, 8, 16, 32}
Random	La, LN	PP	{4, 8, 16, 32}
Adaptive	La, LN	PP	{4, 8, 16, 32}
name	La, LN	PP	{4, 8, 16, 32}
Our-SVM-LNS	La, LN	PP	{4, 8, 16, 32}
Our-Neural-LNS	La, LN	PP	{4, 8, 16, 32}
Our-Bandit-LNS	La, LN	PP	{2, 4, 8, 16, 32} by second arm
Ori-SVM-LNS	PP, PPS, EECBS	PP	Uniformly select from 5 to 16
Ori-Neural-LNS	PP, PPS	PBS	{10, 25, 50} for different maps
Ori-Bandit-LNS	PP, PPS, EECBS	PP	{2, 4, 8, 16, 32} by second arm

Table 2: Number of agents evaluated in different maps

Map	Agent number
empty-32-32 (empty)	300, 350, 400, 450, 500
random-32-32-20 (random)	150, 200, 250, 300, 350
warehouse-10-20-10-2-1 (warehouse)	150, 200, 250, 300, 350
ost003d	200, 300, 400, 500, 600
den520d	500, 600, 700, 800, 900
Paris_1_256 (Paris)	350, 450, 550, 650, 750

3 Neural-LNS Experiment Details

3.1 Environment and Model Hyperparameters

Training We train Neural (NNS) on a cluster equipped with A40 GPUs and Intel Xeon Gold 5218 CPUs. We search for the optimal learning rate within {0.1, 0.01, 0.001, 0.0001, 0.00001} and ultimately select 0.00001. This is smaller than the 0.0001 learning rate used in the original paper because we start from a different initial solution, and we find that a lower learning rate results in a more stable reduction in loss in our unified setting. We use a batch size of 16 and train the model for 10,000 to 100,000 steps until the loss and validation score no longer improve for an additional 1,000 steps. The entire training process is completed in less than 24 hours.

Testing For all experiments, including all rule-based and learning-based methods, we used an Intel E5-2683 CPU with a memory limit of 2G. We use an NVIDIA P100 GPU for Our-Neural and Ori-Neural inference. The average GPU inference overhead of NNS is summarized in Table 3.

Table 3: **Average GPU Overhead for Neural.** The table displays the average GPU overhead for various numbers of agents and map settings across all network forward rounds.

Map	n	Overhead	Map	n	Overhead	Map	n	Overhead
empty	300	0.016	random	150	0.014	warehouse	150	0.042
	350	0.017		200	0.015		200	0.043
	400	0.019		250	0.021		250	0.043
	450	0.022		300	0.024		300	0.044
	500	0.028		350	0.026		350	0.044
ost003d	200	0.020	den520d	500	0.033	Paris	350	0.038
	300	0.020		600	0.041		450	0.038
	400	0.024		700	0.041		550	0.038
	500	0.024		800	0.055		650	0.040
	600	0.027		900	0.045		750	0.042

3.2 Training Data

Original Neural-LNS We used the number of agents, rule-based strategies and neighborhood size suggested by the Neural authors to collect data for the training set. We run 25 to 50 iterations for each map, continuing until there is no further decrease in delay for an additional 5 iterations.

In each iteration, 100 neighborhood candidates are proposed using the suggested rule-based strategy and neighborhood size, and the best neighborhood is selected. The ground truth ranking information for these 100 neighborhood candidates is determined by the delay improvement if each neighborhood is removed and replaced. We attempt to train the model with varying amounts of data, ranging from 1,000 to 9,000 in increments of 1,000, and then select the model that achieves the best performance with the least training data. The exact number of iterations, scenes, neighborhood sizes, and removal strategies used for each map are summarized in Table 4.

Table 4: **Training Data Collection Settings for Ori-Neural-LNS.** 'NS' denotes the size of neighborhood. Iteration denotes the MAPF-LNS iterations. NNS gets one supervised data point per MAPF-LNS iteration, consisting of a MAPF-LNS state and a ground truth ranking of 100 neighborhood candidates generated under that state. Therefore, the total data amount is equals the number of iterations multiplied by the number of scenes.

Ori-Neural-LNS					
Map	Strategy	NB	Iteration	Scenes	Data Amount
empty	Random	50	50	5000	250000
random	RandomWalk	25	50	5000	250000
warehouse	RandomWalk	25	25	5000	125000
ost003d	RandomWalk	10	25	1000	25000
den520d	RandomWalk	25	50	5000	250000
Paris	RandomWalk	25	50	4000	200000

Our Neural-LNS We use a medium number of agents and the best rule-based strategy and neighborhood size for each map under medium agent number to collect training data. The ground truth is defined in the same way as in the Original Neural-LNS. The replan solver is PP, which typically requires more iterations than PBS to converge. As a result, we use fewer scenes per map to gather a comparable amount of data. The exact number of iterations, scenes, neighborhood sizes, and removal strategies for each map are detailed in Table 5.

Table 5: Training Data Collection Settings for Ours-Neural-LNS

Ours-Neural-LNS					
Map	Strategy	NB	Iteration	Scenes	Data Amount
empty	Adaptive	8	1400	300	420000
random	RandomWalk	8	1000	100	100000
warehouse	Adaptive	32	200	200	40000
ost003d	RandomWalkProb	16	400	250	100000
den520d	RandomWalkProb	16	500	200	100000
Paris	RandomWalkProb	32	200	350	70000

3.3 Validation Data

Original Neural-LNS The number of agents, rule-based strategies, neighborhood size, iterations, and ground truth definition are the same as those used for collecting training data in Original Neural-LNS. We run an additional 25 scenes to gather validation data, selecting the best neighborhood in each iteration.

Our Neural-LNS The number of agents, rule-based strategies, neighborhood size, iterations, and ground truth definition are the same as those used for collecting training data in Our Neural-LNS. We use an additional 4 scenes to collect validation data. Similar to training (discussed in Sec 3.2), we use fewer scenes than Original Neural-LNS because Our Neural-LNS uses PP as the replan solver, allowing us to run significantly more iterations and generate more data from a single scene.

3.4 Training procedure

For each map, the model is trained on the corresponding training set. We stop the training when the loss converges and the average ranking on the validation set no longer improves for another 1,000 steps. We calculate the average rank on the validation set to select the best model checkpoint for inference. Here, 'average ranking' means the mean ranking of the best neighborhood predicted by the model appearing in the ground truth ranking over the validation dataset.

4 SVM-LNS Experiment Details

4.1 Environment and Model Hyperparameters

Training We train SVM the model on Intel Xeon Gold 5218 CPUs for 100 iterations.

Testing As described in Sec 3.1, we used an Intel E5-2683 CPU with a memory limit of 2G for inference.

4.2 Training Data

Original SVM-LNS We use the suggested agent numbers by Huang et al. (2022) for training if the map exists in the original paper (i.e. 100 for warehouse and ost003d, 200 for den520d and 250 for Paris). For maps not evaluated in Huang et al. (2022), we use 300 agents for empty-32-32 and 150 agents for random-32-32-20. The neighborhood size is uniformly selected between 5 and 16 (as shown in Table 2). Following the original paper, we run 16 scenes on each map and switch between RandomWalk and

Intersection with equal probability to generated 20 neighborhood candidates. The ground truth ranking information for these 20 candidates is determined by the delay improvement if each neighborhood is removed and replaced.

Our SVM-LNS We use a medium number of agents, the best rule-based strategy, and the best neighborhood size for each map to collect training data (this is the same as the settings for our Neural-LNS, see first three columns in Table 5). Other aspects of the training data collection are the same as in Original SVM-LNS.

Validation Data The validation data collection setup, i.e., agent number, rule-based strategy, and neighborhood size, for Original SVM-LNS and Our SVM-LNS is the same as the training data collection procedure for Original SVM-LNS and Our SVM-LNS, respectively. We run Original SVM-LNS and Our SVM-LNS on 4 additional scenes of each map for 100 iterations to collect validation data. In each iteration, the best neighborhood is selected.

Training Procedure SVM-LNS employs an imitation learning training procedure, collecting training data while updating the model. As described in Sec 4.2, training is performed on 16 scenes, meaning 16 new data points are collected per MAPF-LNS iteration. The model is trained for 100 iterations. The best model is selected based on the average rank on the validation set.

5 Additional Result

Table 6, Table 7, Table 8, and Table 9 show the complete results with runtime limit 300 seconds using LNS2 and LaCAM2 as the initial solvers under all agent number settings.

References

- Erdmann, M.; and Lozano-Perez, T. 1987. On multiple moving objects. *Algorithmica*, 2: 477–521.
- Huang, T.; Li, J.; Koenig, S.; and Dilkina, B. 2022. Anytime multi-agent path finding via machine learning-guided large neighborhood search. In *Proc. AAAI Conf. Artif. Intell. (AAAI)*, volume 36, 9368–9376.
- Li, J.; Chen, Z.; Harabor, D.; Stuckey, P. J.; and Koenig, S. 2021. Anytime multi-agent path finding via large neighborhood search. In *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 4127–4135.
- Li, J.; Chen, Z.; Harabor, D.; Stuckey, P. J.; and Koenig, S. 2022. MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search. In *Proc. AAAI Conf. Artif. Intell. (AAAI)*, volume 36, 10256–10265.
- Li, J.; Ruml, W.; and Koenig, S. 2021. EECBS: A bounded-suboptimal search for multi-agent path finding. In *Proc. AAAI Conf. Artif. Intell. (AAAI)*, volume 35, 12353–12362.
- Okumura, K. 2023. Improving LACAM for Scalable Eventually Optimal Multi-Agent Pathfinding. In *Proc. Int. Joint Conf. Artif. Intell. (IJCAI)*, 243–251.
- Sajid, Q.; Luna, R.; and Bekris, K. 2012. Multi-agent pathfinding with simultaneous execution of single-agent primitives. In *Proc. Int. Symp. Combinatorial Search (SoCS)*, volume 3, 88–96.

Table 6: Final delays of different methods with best neighborhood size, evaluated on maps with differing numbers of agents within 300 seconds. Initial solutions are provided by LNS2. Highlighted are the results ranked **first**, and **second**.

map	n	RNWLK	INTC	RAND	ADP	RWP	Our-S	Our-N	Bandit	Ori-S	Ori-N
empty	300	358.0	406.4	435.1	369.0	369.9	381.5	724.6	386.3	395.6	325.9
	350	750.9	814.6	922.4	770.1	769.0	800.7	1209.9	811.5	807.7	734.3
	400	1397.2	1513.7	1703.2	1418.6	1431.1	1588.3	1928.5	1537.2	1640.4	1464.8
	450	2551.0	2695.0	2908.1	2577.6	2585.3	2766.0	3136.3	2753.7	2936.5	2876.6
	500	4050.5	4205.4	4438.9	4093.8	4051.3	5053.2	4803.2	4318.5	4776.3	5305.6
random	150	332.6	352.6	357.2	330.1	337.4	331.4	360.5	330.1	338.8	323.3
	200	762.0	811.2	831.1	771.3	784.4	786.5	900.2	779.1	790.8	728.5
	250	1504.5	1582.6	1606.3	1527.5	1534.4	1713.9	1683.7	1507.3	1662.8	1528.0
	300	2687.7	2839.7	2885.9	2737.2	2777.2	2885.9	3045.2	2746.0	3005.0	3207.8
	350	4439.2	4609.3	4635.2	4432.8	4408.2	4800.2	5466.7	4564.1	5105.2	6004.1
warehouse	150	117.2	123.5	122.6	109.0	113.0	114.1	244.7	107.9	112.7	164.7
	200	244.2	317.7	286.6	242.8	252.1	245.3	469.6	239.4	252.6	269.1
	250	433.1	695.2	537.7	435.6	443.8	439.6	749.0	414.2	441.0	417.4
	300	663.9	1161.5	950.3	683.7	729.3	681.3	1501.2	669.5	703.2	664.8
	350	1041.8	1949.2	1515.9	1073.0	1134.9	1107.4	1871.1	1047.6	1100.9	1094.5
ost003d	200	150.8	290.7	194.0	149.2	147.6	148.6	165.0	158.2	155.0	245.4
	300	327.3	929.5	632.0	337.6	298.0	338.4	532.8	338.2	338.2	582.6
	400	761.8	2048.3	1668.8	746.1	652.3	850.3	679.5	1276.3	1086.3	1483.2
	500	2051.4	4460.2	3611.6	2094.8	1515.2	2928.2	2012.2	3059.8	3092.7	4673.8
	600	6069.5	8824.9	8424.3	6325.9	4731.3	10104.8	5501.2	6093.6	8604.1	11426.4
den520d	500	293.5	1414.3	906.1	306.2	248.5	252.5	313.5	607.8	394.3	732.3
	600	536.9	2293.2	1656.1	583.1	396.7	389.0	496.1	1247.0	972.6	1786.5
	700	934.1	3415.0	2907.4	1049.8	620.2	665.2	814.0	2297.3	1880.1	4228.7
	800	1476.1	4535.4	4639.0	1685.4	883.2	1003.4	1102.0	3330.2	3876.9	7766.6
	900	2290.1	6776.0	6447.9	2611.1	1387.2	1816.6	1821.4	5343.4	6161.2	13367.1
Paris	350	80.0	121.5	83.8	75.5	76.4	71.9	78.6	1066.5	81.7	193.5
	450	127.4	273.6	172.7	124.9	118.8	120.7	142.6	737.1	123.2	130.8
	550	203.6	540.8	345.4	192.2	183.8	184.7	205.3	721.5	196.5	721.5
	650	278.1	963.6	650.2	270.2	257.1	262.0	313.1	1022.0	286.8	307.5
	750	404.7	1680.8	1211.7	396.9	375.6	388.7	468.2	576.9	483.2	2409.4

Table 7: AUC (divided by 10k) of different methods with best neighborhood size, evaluated on maps with differing numbers of agents within 300 seconds. Initial solutions are provided by LNS2. Highlighted are the results ranked **first**, and **second**.

map	n	RNWLK	INTC	RAND	ADP	RWP	Our-S	Our-N	Bandit	Ori-S	Ori-N
empty	300	12.2	13.8	15.2	12.6	12.6	15.4	26.9	13.3	18.2	12.9
	350	25.6	28.6	31.5	26.5	27.7	32.9	42.9	28.5	32.8	28.9
	400	49.1	53.7	59.6	50.0	50.1	64.5	70.0	54.2	63.9	56.8
	450	90.5	95.4	101.8	90.7	91.1	105.0	112.2	96.5	109.9	107.5
	500	145.39	143.9	155.5	140.4	143.9	167.1	179.7	150.8	174.7	186.4
random	150	10.5	11.4	11.3	10.4	10.8	10.6	11.9	10.3	11.0	10.4
	200	25.4	26.2	27.0	25.6	25.4	27.3	30.4	24.9	26.3	24.2
	250	50.4	53.2	53.3	50.6	50.9	59.1	57.5	49.3	58.3	54.8
	300	93.8	98.6	100.1	93.9	94.6	104.9	108.4	92.0	107.0	115.5
	350	150.4	155.5	167.8	150.1	156.6	175.8	193.5	155.8	186.3	206.3
warehouse	150	3.7	7.0	4.8	3.6	3.6	4.6	8.9	3.6	4.0	5.5
	200	8.1	15.6	12.2	8.4	8.7	9.7	17.7	8.0	9.8	10.5
	250	15.3	32.8	23.4	16.2	16.4	19.3	31.4	14.1	21.1	17.7
	300	25.6	53.7	42.0	29.1	27.9	34.3	57.3	23.8	36.8	31.1
	350	41.3	84.6	66.2	44.1	44.7	65.7	77.1	38.1	64.6	56.1
ost003d	200	6.8	14.4	9.9	6.3	5.5	6.6	6.6	8.3	9.9	12.0
	300	19.9	45.6	36.5	21.3	13.9	23.9	15.8	33.4	34.3	39.2
	400	48.3	100.6	86.3	51.0	39.3	84.3	41.2	75.2	106.5	109.0
	500	142.9	223.1	193.7	141.0	103.4	217.0	121.8	163.6	223.4	270.5
	600	318.3	404.8	399.2	323.0	260.7	459.1	294.6	311.4	457.2	517.9
den520d	500	22.8	72.4	57.9	26.1	14.1	28.0	20.4	47.8	60.7	87.8
	600	44.4	116.0	101.5	50.2	26.5	46.3	37.0	89.2	117.2	173.4
	700	80.3	175.9	171.7	88.0	48.3	77.9	68.2	150.2	200.1	286.7
	800	123.0	234.8	258.2	131.9	75.4	142.8	99.7	209.4	312.9	431.3
	900	166.7	347.7	345.6	187.9	130.6	216.5	168.1	308.8	420.2	624.8
Paris	350	2.9	9.4	6.5	3.3	2.7	6.3	10.7	4.6	7.4	38.9
	450	4.9	22.2	17.2	6.6	4.5	12.2	7.1	11.0	20.3	42.9
	550	9.1	38.3	33.7	11.7	7.4	16.5	12.2	20.7	37.6	67.7
	650	14.0	66.7	57.1	17.7	12.2	35.7	18.7	37.0	78.2	128.7
	750	25.4	110.9	99.1	33.7	21.8	65.3	36.6	70.6	126.6	243.5

Table 8: Final delays of different methods with best neighborhood size, evaluated on maps with differing numbers of agents within 300 seconds. Initial solutions are provided by LaCAM2. Highlighted are the results ranked **first**, and **second**.

map	n	RNWLK	INTC	RAND	ADP	RWP	Our-S	Our-N	Bandit	Ori-S	Ori-N
empty	300	350.5	402.1	441.9	369.0	368.1	374.8	790.5	389.3	397.0	371.0
	350	740.3	824.2	930.2	758.3	759.4	823.0	1296.4	844.4	880.2	826.4
	400	1447.9	1593.4	1756.2	1476.5	1484.4	1692.4	2094.8	1579.0	1857.5	1614.3
	450	2638.0	2779.5	3040.7	2684.1	2681.5	3020.0	3598.3	2828.1	3506.0	3318.3
	500	4237.7	4448.5	4828.2	4374.5	4335.7	5194.9	5808.0	4772.6	5793.6	6085.3
random	150	327.4	344.3	348.4	324.6	331.0	332.9	361.7	326.5	337.6	330.2
	200	751.1	805.8	836.8	770.6	767.8	780.8	887.4	732.4	777.5	770.6
	250	1472.7	1537.2	1613.5	1506.7	1494.1	1521.2	1747.1	1456.3	1604.8	1698.5
	300	2547.4	2769.2	2811.8	2622.6	2663.6	2800.1	3171.0	2674.4	3084.9	3435.9
	350	4341.6	4622.2	4683.9	4406.5	4367.5	5162.7	5886.5	4598.5	5983.6	7438.8
warehouse	150	114.0	127.6	121.3	111.4	112.8	112.0	188.7	110.3	114.8	206.5
	200	247.5	309.6	281.0	244.2	249.1	248.8	420.8	239.4	250.6	375.5
	250	430.0	698.5	562.4	424.9	446.7	444.6	782.1	426.2	445.1	961.5
	300	680.6	1264.3	973.9	673.7	703.5	673.7	1423.9	695.9	716.2	2248.7
	350	1043.0	2011.0	1509.0	1055.0	1130.8	1042.4	1907.0	1067.6	1097.6	5255.1
ost003d	200	155.6	299.4	183.3	153.3	147.4	147.5	162.3	170.1	154.7	496.8
	300	333.8	868.4	611.8	333.4	296.1	297.9	339.3	496.5	359.8	1698.9
	400	714.7	1849.6	1488.2	805.2	622.9	762.5	675.3	1253.3	1083.7	5277.4
	500	2341.8	4338.9	3900.2	2067.5	1637.6	4386.8	2379.1	3023.9	3719.0	12525.6
	600	7514.0	10394.8	12466.1	8216.9	5875.0	13066.6	8469.1	7244.6	10153.9	25804.5
den520d	500	311.1	1433.0	972.7	311.5	241.6	240.1	322.6	764.6	440.5	3437.9
	600	546.3	2289.3	1649.3	597.4	386.6	381.0	513.4	1235.4	1353.4	8231.3
	700	933.1	3163.0	2782.7	1075.2	622.4	660.7	816.6	2439.2	2259.0	15529.0
	800	1548.6	4437.5	4337.2	1574.1	911.5	1003.6	1323.8	3671.6	4906.7	21966.5
	900	2486.1	6426.0	6923.5	2819.9	1382.0	2058.6	2067.1	5595.4	15199.0	31694.0
Paris	350	87.4	130.6	84.4	78.1	75.8	79.0	181.7	83.6	80.0	1285.3
	450	132.5	274.9	174.6	123.3	118.4	119.7	144.4	133.4	133.0	1483.0
	550	198.0	776.5	466.9	194.8	183.3	184.7	228.9	210.2	200.6	2643.7
	650	276.7	960.1	758.5	275.4	257.5	260.6	319.2	337.4	319.8	7697.9
	750	373.2	1821.2	1362.6	385.8	366.0	369.9	498.8	627.3	616.3	13831.5